SYSTEM DESIGN & ARCHITECTURE DOCUMENT

NUVI Social Media Advertisement Distribution Manager

A simple, lightweight design for a Chrome extension. It has no server or database; all data stays in the user's browser, and the user stays in control of every Facebook post.

BASED ON

BRD v1.0

13 November 2026 · with Section 14 defaults

PREPARED BY

Amritha Hitige

NUVI Engineering (Pvt) Ltd

PLATFORM

Google Chrome Extension (MV3)

Side Panel · Service Worker · Local Storage

DATE / VERSION

13 November 2026 · v1.0

Draft for review

CONTENTS

01 Purpose & Design Principles 2

02 Architecture Overview 3

03 Components 4

04 Project Structure & Manifest 5

05 Data Design 6

06 Distribution State Machine 7

07 Key Flows 8

08 Messaging & Tab Handling 9

09 Error Handling & Recovery 10

10 Security & Privacy 10

11 Requirements Traceability 11

12 Build, Test, Install & Future 12

01 · PURPOSE & DESIGN PRINCIPLES

A small application, kept small on purpose

This document explains **how** the Distribution Manager is built: its parts, where data is kept, how a distribution campaign runs, and how the system recovers when Chrome closes. The **what** is defined in the BRD.

Design principle	What it means in practice
1. Local-first, no server	Everything runs inside Chrome. No backend, no cloud database, no login. Nothing to host and no running cost.
2. User stays in control	The extension only **navigates** a tab. It never reads, changes or clicks anything on Facebook (BR-008, BR-015).
3. Save after every action	The campaign state is written to storage after every click, so closing Chrome never loses progress (BR-010).
4. One owner for distribution logic	Only the background service worker changes campaign progress. The UI asks it to act, which avoids conflicting updates.
5. Minimum permissions	No host permissions and no content scripts. Only storage, side panel, unlimited storage and clipboard write.
6. Plain technology	HTML, CSS and vanilla JavaScript (ES modules). No build tools are required.
7. Business separation by design	Every group, advertisement and campaign carries a `businessId`, and every query filters by the selected business (BR-013).

Scope of this document

Covered
Architecture, components, data design, state machine, main flows, messages, tab handling, error recovery, security, traceability, and build and install.

Not covered
Screen visual design (see BRD Section 7), detailed code, and Phase 3 analytics. The Technical Document will add field-level rules and test cases.

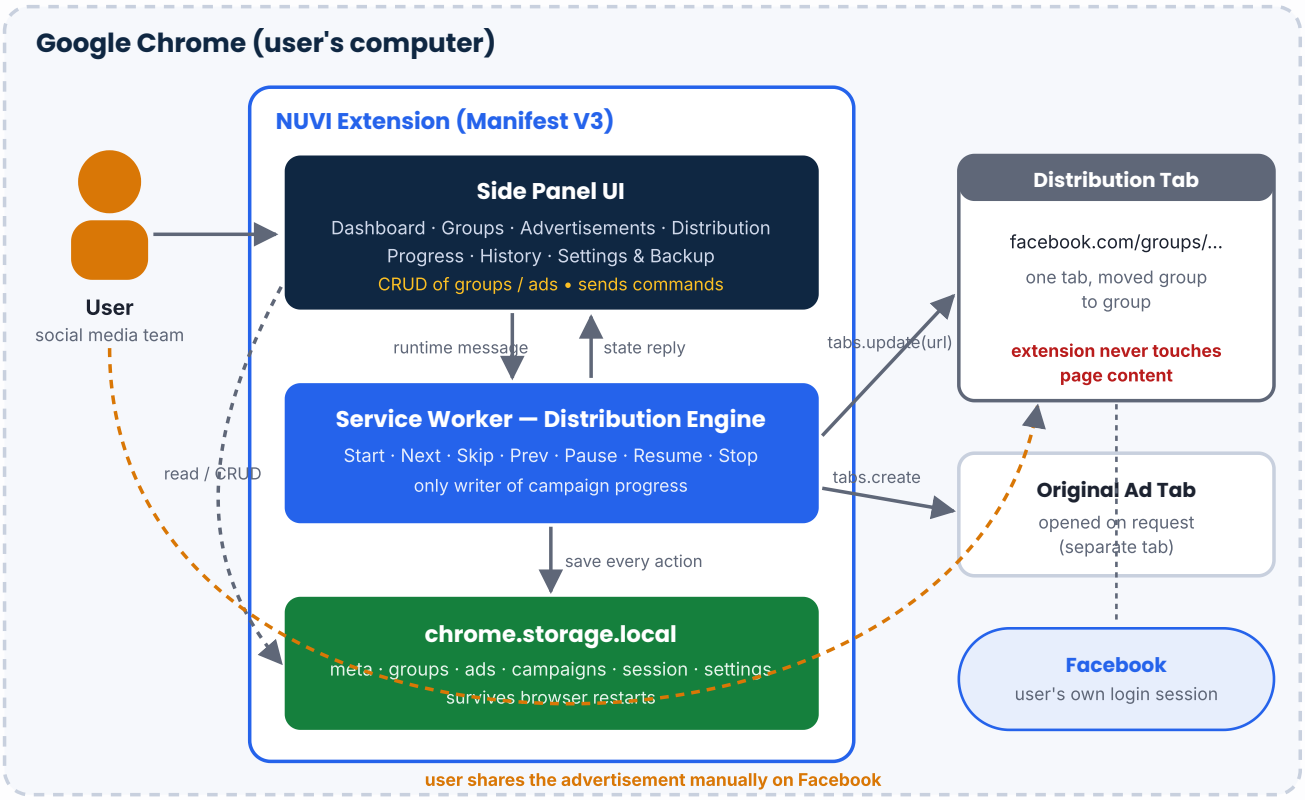
Assumptions taken from the BRD (Section 14 defaults)

#	Assumption
1	One main user on one computer and Chrome profile; data moves only by backup and restore.
2	The **Chrome Side Panel** is the main user interface (not a popup).
3	Sharing is done by "Copy Ad Link" (paste into a group post) or "Open Original" in a separate tab.
4	The posting-gap timer is optional and comes in Phase 2.
5	Internal installation (unpacked or private package), not the public Chrome Web Store.

02 · ARCHITECTURE OVERVIEW

The whole system on one page

There are three parts inside the extension (UI, background engine, storage) and one Chrome tab that shows Facebook. No other server is involved.



Part	Responsibility
Side Panel UI	All screens. Adds, edits and deletes groups and advertisements directly in storage. For distribution, it only **sends commands** and shows the state.
Service Worker	The distribution engine: owns the active session, moves the distribution tab, records statuses and finishes campaigns.
chrome.storage.local	The single source of truth for all data. The UI listens to `storage.onChanged` to refresh automatically.
Distribution Tab	A normal Chrome tab that the engine moves from group to group. The user posts there manually.

03 · COMPONENTS

Building blocks and what each one does

Component	Responsibility	BRD
Side Panel (sidepanel/)		
app.js (router)	Loads views, keeps the selected business, applies business colour, shows the resume banner	FR-001–003, 025
views/dashboard	Totals and menu	FR-002
views/groups	List, search, filter, add/edit/delete, CSV import preview, CSV export	FR-004–010, 030–031
views/ads	List, add/edit, archive/delete, per-ad distribution view	FR-011–015, 040
views/distribution	Choose advertisement and groups, duplicate markers, Start	FR-016–018, 026
views/progress	Current group, counters, action buttons, Copy Ad Link, Open Original, summary	FR-019–024, 032–037
views/history	Campaign list and detail, reuse, later status update	FR-027–029, 038
views/settings	Drop-down lists, posting gap, backup / restore, reset	FR-039, 041–043
Background (background/)		
service-worker.js	Entry point: opens the side panel on icon click, routes messages to the engine	FR-032
distribution-engine.js	State machine; creates campaigns; Next / Skip / Prev / Pause / Resume / Stop; finishes campaigns	FR-018–025, 036–037
tab-manager.js	Opens or moves the distribution tab; re-creates it if it was closed; opens the original ad tab	FR-019, 021, 033
Shared (shared/) – used by both		
storage.js	Small repository layer: `get / save / remove` per collection; schema version and migrations	NFR-03, 06
url-utils.js	Validates Facebook group and post URLs; normalizes URLs for duplicate checks	FR-015, 030
csv.js	Parses and creates UTF-8 CSV (quoted fields, Sinhala text), validates rows	FR-010, 031
backup.js	Exports all data to a JSON file and restores it with validation	FR-041–042
ids.js / constants.js	ID generation (GRP-, AD-, CAM-), businesses B001/B002, status names, colours	FR-001, BR-001

Who writes what: the **UI** writes groups, ads and settings. The **service worker** writes the session and campaign progress. Neither writes the other's data, so the two never overwrite each other.

Folders and the extension manifest

```
nuvi-distribution-manager/
├─ manifest.json
├─ background/
│  ├─ service-worker.js
│  ├─ distribution-engine.js
│  └─ tab-manager.js
├─ sidepanel/
│  ├─ index.html
│  ├─ app.js
│  ├─ styles.css
│  └─ views/
│     ├─ dashboard.js
│     ├─ groups.js
│     ├─ ads.js
│     ├─ distribution.js
│     ├─ progress.js
│     ├─ history.js
│     └─ settings.js
├─ shared/
│  ├─ storage.js
│  ├─ url-utils.js
│  ├─ csv.js
│  ├─ backup.js
│  ├─ ids.js
│  └─ constants.js
├─ icons/  (16, 32, 48, 128 px)
├─ samples/
│  └─ groups-template.csv
└─ tests/  (unit tests for shared/)
```

```
{
  "manifest_version": 3,
  "name": "NUVI Social Media Distribution Manager",
  "version": "1.0.0",
  "description": "Guided Facebook group distribution
                  for NUVI Engineering and NUVI IT",
  "minimum_chrome_version": "116",
  "permissions": [
    "storage",
    "unlimitedStorage",
    "sidePanel",
    "clipboardWrite"
  ],
  "background": {
    "service_worker": "background/service-worker.js",
    "type": "module"
  },
  "side_panel": {
    "default_path": "sidepanel/index.html"
  },
  "action": {
    "default_title": "NUVI Distribution Manager"
  },
  "icons": { "128": "icons/icon128.png" }
}
```

Permission	Why it is needed
`storage`	Save all data locally
`unlimitedStorage`	Remove the 10 MB limit for long history
`sidePanel`	Keep the UI open beside Facebook
`clipboardWrite`	"Copy Ad Link" button

No `host_permissions`, no `content_scripts` and no `"tabs"` permission. Creating and updating a tab's URL doesn't need them, because the extension never reads Facebook page content or tab URLs. Chrome shows no "read and change your data on all websites" warning.

05 · DATA DESIGN

What is stored, and where

All data lives in `chrome.storage.local` under six keys. Each collection is an object keyed by ID, so lookups are fast and simple.

Storage key	Contents	Written by
`meta`	Schema version, ID counters, last backup date, last selected business	UI + engine
`groups`	All Facebook groups of both businesses	UI
`ads`	All master advertisements	UI
`campaigns`	Every campaign with its group results (history)	Engine (UI only for FR-038)
`session`	The single active campaign pointer (null when idle)	Engine
`settings`	Drop-down lists, posting gap, backup reminder days	UI

```
// groups["GRP-0001"]
{
  "id": "GRP-0001",
  "businessId": "B001",
  "name": "Kiriwaththuduwa Api",
  "url": "https://www.facebook.com/
         groups/233212924715137/",
  "urlKey": "facebook.com/groups/
            233212924715137",
  "category": "Location-Based",
  "location": "Kiriwaththuduwa",
  "status": "ACTIVE",
  "notes": "Post link only",
  "createdAt": "2026-11-13T09:10:00Z",
  "updatedAt": "2026-11-13T09:10:00Z"
}

// ads["AD-0001"]
{
  "id": "AD-0001",
  "businessId": "B001",
  "name": "Kiriwaththuduwa House for Sale",
  "postUrl": "https://www.facebook.com/...",
  "type": "House",
  "description": "Two-bedroom house",
  "status": "ACTIVE",
  "createdAt": "2026-11-13T09:15:00Z"
}
```

```
// campaigns["CAM-004"]
{
  "id": "CAM-004",
  "businessId": "B001",
  "adId": "AD-0001",
  "adName": "Kiriwaththuduwa House for Sale",
  "adUrl": "https://www.facebook.com/...",
  "status": "IN_PROGRESS",
  "currentIndex": 2,
  "startedAt": "2026-11-13T09:20:00Z",
  "completedAt": null,
  "items": [
    { "groupId": "GRP-0001",
      "groupName": "Kiriwaththuduwa Api",
      "groupUrl": "https://...",
      "status": "PUBLISHED",
      "reason": null,
      "log": [{"s":"IN_PROGRESS","at":"..."},
              {"s":"PUBLISHED","at":"..."}] },
    { "groupId": "GRP-0002", "status": "SKIPPED" },
    { "groupId": "GRP-0007",
      "status": "IN_PROGRESS" }
  ]
}

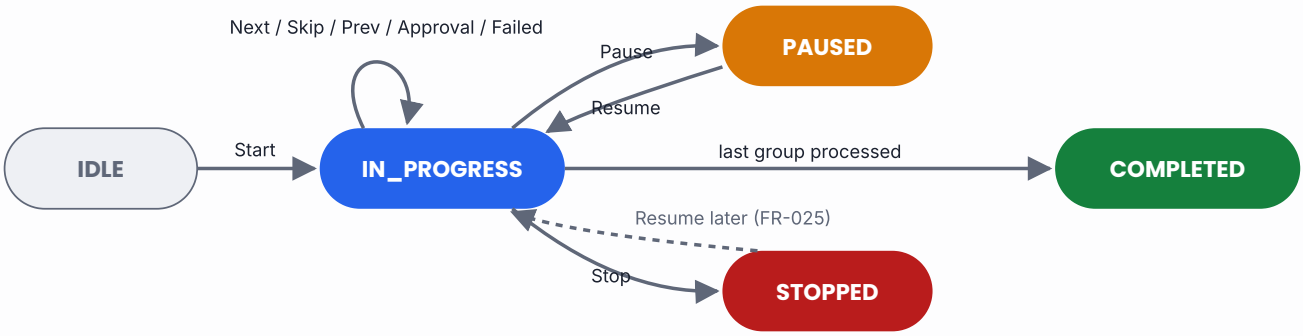
// session
{ "campaignId": "CAM-004",
  "tabId": 1834,
  "updatedAt": "2026-11-13T09:31:00Z" }
```

Design decision	Reason
Name and URL snapshots in campaigns	History stays correct even if a group or advertisement is edited or deleted later (BR-017)
`urlKey` (normalized URL)	Fast duplicate detection within a business (FR-030, BR-016)
Size estimate	500 groups ≈ 250 KB; 1,000 campaigns × 40 groups ≈ 8 MB, so `unlimitedStorage` is a safety margin

06 · DISTRIBUTION STATE MACHINE

How a campaign and each group change state

Campaign states



Campaign state	Meaning	Allowed actions
IDLE	No active session (`session = null`)	Start
IN_PROGRESS	A group is open; waiting for the user's result	Published-Next, Skip, Approval Required, Failed, Previous, Pause, Stop
PAUSED	Temporarily stopped; the tab is not moved	Resume, Stop
STOPPED	Ended by the user; unprocessed groups stay Pending; session cleared	Resume from History (re-opens as IN_PROGRESS)
COMPLETED	Every group has a final status; summary shown; session cleared	View summary, reuse (creates a new campaign)

Group (item) states inside a campaign

From	Action	To
PENDING	Engine opens this group in the distribution tab	IN_PROGRESS
IN_PROGRESS	"Published – Next Group"	**PUBLISHED**
IN_PROGRESS	"Skip"	SKIPPED
IN_PROGRESS	"Approval Required" (+ optional reason)	APPROVAL_REQUIRED
IN_PROGRESS	"Failed" (+ optional reason)	FAILED
IN_PROGRESS	"Previous": the current item goes back to PENDING, and the previous item re-opens	PENDING
Any final status	Later correction from History (FR-038), logged with date	Any final status

Rule built into the engine: no code path sets PUBLISHED except the user's "Published – Next Group" click or a manual History correction. Opening a page never changes the status to Published (BR-009).

07 · KEY FLOWS

Step-by-step flows

A. Start distribution

#	From → To	Action
1	User → Side Panel	Select advertisement and groups, then click START
2	Side Panel	Check against history; show duplicate markers or warnings (FR-026)
3	Side Panel → Engine	`{type:"START", adId, groupIds}`
4	Engine → Storage	Create campaign CAM-nnn (snapshots, all items PENDING), write `session`
5	Engine → Tab Manager	Use the current tab as the distribution tab; set item 0 to IN_PROGRESS; `tabs.update(tabId,{url})`
6	Storage → Side Panel	`storage.onChanged` triggers the switch to the Progress view

B. Published – Next Group (same pattern for Skip, Approval Required, Failed)

#	From → To	Action
1	User	Shares the advertisement on Facebook manually, then clicks **Published – Next Group**
2	Side Panel → Engine	`{type:"RESULT", status:"PUBLISHED", reason:null}`
3	Engine → Storage	Save the current item's status and log; move to the next PENDING item; save **before** navigating
4	Engine	If the next group was already published with this ad → ask the panel to confirm (Continue / Skip)
5	Engine → Tab Manager	Move the distribution tab to the next group URL (re-create the tab if it was closed)
6	Engine	If no PENDING items remain → campaign COMPLETED, session cleared, summary shown

C. Recovery after Chrome restart

> The side panel reads `session` on open. If it is not null, a **"Resume CAM-004 (3 / 30)"** banner appears. On Resume, the engine finds the old `tabId` invalid, uses the active tab or creates one, and re-opens the current IN_PROGRESS group.

D. CSV import · E. Backup / Restore

CSV import (UI only)
Read file (UTF-8) → parse rows
Validate each row: required fields, group URL format
Normalize the URL → compare with existing `urlKey` s and the file itself
Show preview: valid / invalid / duplicate
On confirm: save valid rows in one write; show summary

Backup / Restore (UI only)
Backup: read all keys → JSON `{app, schemaVersion, exportedAt, data}` → download `nuvi-backup-YYYY-MM-DD.json`
Restore: blocked while a campaign is IN_PROGRESS
Validate the file → migrate if it uses an older schema
Warn "replace all data?" → write all keys

How the side panel talks to the engine

Uses `chrome.runtime.sendMessage`. Every message returns `{ok:true, state}` or `{ok:false, error}`. The panel also listens to `chrome.storage.onChanged`, so the screen stays correct even if the service worker restarted.

Message	Payload	Engine action
`GET_STATE`	—	Return the active session, campaign and counters
`START`	`adId, groupIds[]`	Create the campaign, open the first group (refused if a session is already active: BR-018)
`RESULT`	`status, reason?`	Save the result for the current group, open the next group
`PREVIOUS`	—	Current item → PENDING, re-open the previous group
`PAUSE` / `RESUME`	— / `campaignId?`	Set the state; on Resume, re-open the current group
`STOP`	—	Mark STOPPED, clear the session
`OPEN_ORIGINAL`	—	Open the advertisement URL in a separate tab (re-use it if already open)
`CONFIRM_DUPLICATE`	`decision: continue | skip`	Continue to the group or mark it Skipped

"Copy Ad Link" is done directly in the side panel with `navigator.clipboard.writeText()`; it needs no message.

Tab handling rules

Situation	Behaviour
Start	The active tab of the current window becomes the distribution tab; its ID is saved in `session.tabId`
Next / Skip / Previous	`chrome.tabs.update(tabId, {url, active:true})`, the same tab every time (FR-019)
User closed the tab	`tabs.update` fails → `tabs.create({url})` → save the new `tabId` (FR-033). Only one new tab is created
User navigated the tab elsewhere	Allowed (e.g. to share from the original post). The next action simply moves the tab to the next group
Open Original	Opened in a **separate** tab; the distribution tab keeps the group page
Chrome restarted	The old tab ID is invalid → handled as "user closed the tab"

Service worker sleep (MV3): Chrome may stop the service worker after about 30 seconds of inactivity. This is harmless here, because every message wakes it and the engine **reloads its state from storage at the start of every message**. It never relies on in-memory variables.

09 · ERROR HANDLING & RECOVERY

What can go wrong, and what the system does

Problem	Handling
Invalid group / post URL entered	Inline message: "Please enter a valid Facebook group link (facebook.com/groups/...)"; save is disabled
Duplicate group URL	"This group already exists: Homagama Api". Not saved
CSV with bad rows	Bad rows listed with the reason; good rows can still be imported
Distribution tab closed	A new single tab is created automatically
Chrome closed mid-campaign	Session kept in storage; Resume banner shown on next open
Group deleted while in a campaign	The campaign uses its URL snapshot; it continues normally
Storage write failure	Action not completed; error toast "Could not save. Please try again."; the tab does not move
Invalid or corrupted backup file	Restore refused; existing data untouched
Start pressed during an active campaign	"CAM-004 is still in progress. Pause or stop it first." (BR-018)
No backup for 7 days	Reminder banner on the dashboard

10 · SECURITY & PRIVACY

Safe by design

Control	How
No credentials	Never asks for or stores Facebook passwords or cookies; uses the browser's existing Facebook login (NFR-04, BR-019)
No page access	No content scripts, no host permissions, no reading of Facebook pages (BR-015)
No network calls	The extension makes no requests to any server; all code is bundled locally
Strict content security	MV3 default CSP: no remote scripts, no `eval`; libraries (if any) are bundled
Safe rendering	User text (group names, notes) is inserted as text (`textContent`), never as HTML
Backup files	Contain only group, ad and campaign data, with no personal account data; the user stores them safely

11 · REQUIREMENTS TRACEABILITY

Where each BRD requirement lives in the design

BRD module	Requirements	Design components
M01 Business	FR-001–003	constants.js (B001/B002), app.js business switch, `businessId` filter in storage.js
M02 Groups	FR-004–010, 030–031	views/groups, url-utils.js (validate, `urlKey`), csv.js
M03 Advertisements	FR-011–015	views/ads, url-utils.js (post URL)
M04 Distribution	FR-016–022, 032–035	views/distribution, views/progress, distribution-engine.js, tab-manager.js, side panel
M05 Progress	FR-023–026, 036–039	Item state machine, session, Resume banner, duplicate check, summary view, posting-gap timer (Phase 2)
M06 History	FR-027–029, 040	`campaigns` key with snapshots, views/history
M07 Settings & Data	FR-041–043	backup.js, views/settings, `meta.lastBackupAt`
Business rules	BR-001–019	Engine guards (BR-007–012, 018), storage design (BR-016–017), no page access (BR-015, 019)
Non-functional	NFR-01–10	See below

Non-functional requirements → design

NFR	Design answer
Performance (500 groups)	Data kept in memory in the panel after one read; search and filter in plain JS; no network
No extra tabs	One distribution tab + one optional original-ad tab (tab-manager.js)
Reliability	Save before navigating; reload state on every message; Resume banner
Security / permissions	4 permissions only; no host access; no network
Compatibility	Chrome 116+ desktop (side panel API)
Maintainability	Small modules; businesses and lists as data; schema version + migrations
Data safety	Backup / restore, 7-day reminder, warning before uninstall (in the docs)

Getting it built and into use

Technology

Language	JavaScript (ES modules)
UI	Plain HTML + CSS (React optional, not required)
Build	None needed; load the folder directly
Tests	Unit tests for `shared/` (URL rules, CSV, backup) and the engine state machine
Source control	NUVI GitHub repository

Installation (internal)

1	Open `chrome://extensions`
2	Turn on **Developer mode**
3	**Load unpacked** → select the folder
4	Pin the icon; clicking it opens the side panel
5	Settings → import the groups CSV

Testing approach

Level	What is tested
Unit	URL validation and normalization, CSV parsing (Sinhala, quotes, duplicates), state transitions, backup round-trip
Manual (UAT)	BRD acceptance criteria AC-001 to AC-022 with the real 39 Engineering groups
Resilience	Close the tab mid-campaign; restart Chrome; wait more than 1 minute (service worker sleep) and then click Next

Updates & versioning

Version numbers follow `MAJOR.MINOR.PATCH`. Any change to stored data increases `schemaVersion` and adds a small migration in `storage.js`, so existing data upgrades automatically. Take a backup before every update.

Future evolution (not in Version 1)

Need	Possible design change
Several team members share data	Add a small backend (e.g. NUVI's standard Spring Boot + PostgreSQL) and sync; `storage.js` becomes the sync layer
Phase 3 analytics and reports	New view reading `campaigns`; CSV / PDF export
More businesses	Move businesses from `constants.js` to storage, with an Add Business screen
Other Chromium browsers	Test on Edge / Brave; the side panel API differs by browser

Summary: one Chrome extension with a side panel, one background engine, one local storage, four permissions, and no server. Simple to build, simple to install, and the user stays in control.